**Università degli Studi di Messina**
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

# HANDS-ON 11:
Meccanismi di Sincronizzazione tra Thread

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

# Indice

# 1 Problem Statement

L'undicesima esercitazione (HO11) è focalizzata sulle problematiche della concorrenza nei paradigmi di programmazione multithreading, in particolar modo sull'individuazione e risoluzione delle *Race Conditions*. L'obiettivo si articola in tre task principali: 1. La realizzazione di un programma in cui 10 thread incrementano un contatore globale condiviso all'interno di una *sezione critica*, senza tuttavia avvalersi di meccanismi di sincronizzazione. 2. La risoluzione della medesima operazione ricorrendo a costrutti per la Mutua Esclusione (*Mutex*). 3. L'implementazione di un differente scenario in cui l'accesso alla sezione critica da parte di 5 thread viene limitato per permettere l'ingresso massimo a due soli thread alla volta, sfruttando l'astrazione concettuale dei Semofori posix a conteggio limitato.

# 2 Stato dell'Arte

Le problematiche sollevate nell'Hands-On 11 costituiscono le fondamenta della programmazione concorrente:

- **Sezione Critica e Condizioni di Corsa**: Si verifica quando più flussi di esecuzione (es. thread) accedono simultaneamente e in modalità non atomica a informazioni condivise (memoria, variabili). La mancanza di atomicità espone le operazioni d'aggiornamento a sovrascritture incrociate al momento della schedulazione del sistema operativo.

- **Mutex (Mutual Exclusion)**: Costituisce il costrutto nativo delle specifiche `Pthreads` (tramite `pthread_mutex_t`) per implementare il blocco (*lock*) granulare all'ingresso della sezione critica e lo sblocco (*unlock*) all'uscita. Garantisce formalmente che solo un thread possa trovarsi all'interno del proprio perimetro in ogni dato istante temporale.

- **Semofori (Counting Semaphores)**: Strumento di generalizzazione (gestito via `<semaphore.h>`). A differenza di un mutex (che è assimilabile spesso a un semaforo binario controllante una sola risorsa esatta), un semaforo inizializzato a $N$ protegge l'accesso limitato ad $N$ entità. I thread usano `wait()` per acquisire un token, causandone decremento, e `post()` per rilasciarlo al termine dell'operazione.

# 3 Metodologia ed Implementazione

Per coprire gli obiettivi prestabiliti, l'ecosistema del laboratorio fa uso della libreria `<pthread.h>` nello standerd POSIX e `<semaphore.h>` per le varianti col semaforo.

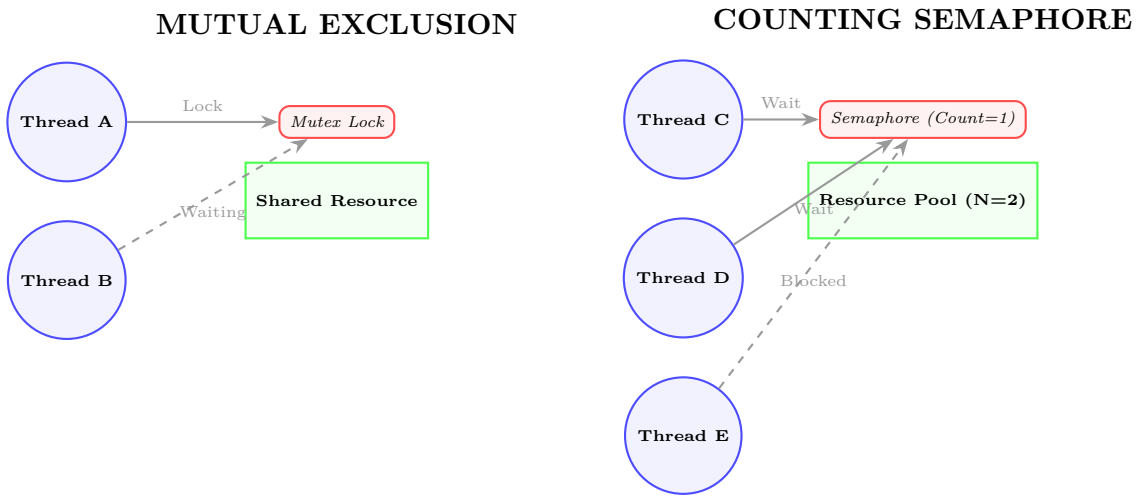


Figure 1: Differenza concettuale tra Mutex (Esecuzione Esclusiva) e Semaforo (Accesso Limitato a pool).

## 3.1 Race Condition Senza Sincronizzazione

Nel primissimo passo viene volutamente violata la sincronizzazione. Il `shared_counter++` viene eseguito da 10 thread 10.000 volte ciascuno.

**Snippet Esercizio 1: Assenza di controllo in Sezione Critica**

```
void* thread\_function(void* arg) {
    long thread\_id = (long)arg;
    for (int i = 0; i < MAX\_VALUE; i++) {
        shared\_counter++; // <--- Non atomico: Read,
            Increment, Write
    }
    printf("Thread %ld: shared\_counter = %d\n", thread\_id,
        shared\_counter);
    pthread\_exit(NULL);
}
```

## 3.2 Mutua Esclusione con Mutex

L'approccio corretto del medesimo codice precedente fa uso del lock della libreria pthread passandogli l'indirizzo di un `counter_mutex`.

**Snippet Esercizio 2: Controllo tramite Mutex**

```
for (int i = 0; i < MAX\_VALUE; i++) {
    pthread\_mutex\_lock(&counter\_mutex);
    shared\_counter++; // <--- Protetto al 100% da accesso multi-thread
    pthread\_mutex\_unlock(&counter\_mutex);
}
```

### 3.3 Gestione della Concorrenza Parziale (Semofori)

Il terzo esercizio introduce l'entità `sem_t` di cui si decreta in fase di inizializzazione un contatore esatto di concorrenza.

**Snippet Esercizio 3: Semaforo max limit 2**

```
sem\_wait(&max\_concurrency\_sem); // Thread attende il token
// ... Critical Section accessibile per due thread contemporanei ...
sleep(2); // Lavoro pesante simulato
sem\_post(&max\_concurrency\_sem); // Ritiro e rilascio
```

Il cui setup del semaforo poggia sulla chiamata `sem_init(&max_concurrency_sem, 0, 2);`.

### 3.4 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

**Build Run**

```
$ make
$ ./programma
```

## 4 Risultati

L'esecuzione della pipeline ha restituito un comportamento pienamente conforme all'osservazione teorica.

### 4.1 Output Senza Sincronizzazione

**Output di esecuzione ex1_no_sync**

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho11$
   ./ex1\_no\_sync
Thread 9: shared\_counter = 76935
Final value of shared\_counter: 76935 (Expected: 100000)
```

La chiara denotazione di "informazione perduta" riflette statisticamente che le operazioni non sono avvenute correttamente.

### 4.2 Output con Sincronizzazione Mutex

**Output di esecuzione ex2_sync**

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho11$
   ./ex2\_sync
Thread 4: shared\_counter = 100000
Final value of shared\_counter: 100000 (Expected: 100000)
```

Con un perimetro sincronizzato l'incremento di ogni esecuzione finisce esattamente alla cifra teoricamente attesa di 100.000 (10.000 incremento a thread $\times$ 10 thread allocati).

### 4.3 Output Multi-Accesso Semaforo

**Output di esecuzione ex3_semaphore**

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho11$
   ./ex3\_semaphore
Thread 0 has ENTERED the critical section.
Thread 1 has ENTERED the critical section.
Thread 2 is waiting to enter the critical section...
Thread 4 is waiting to enter the critical section...
Thread 3 is waiting to enter the critical section...
Thread 0 is LEAVING the critical section.
Thread 1 is LEAVING the critical section.
Thread 2 has ENTERED the critical section.
```

Solo all'uscita delle coppie 0-1 scatta lo sblocco in grado di far defluire il Thread 2 rimasto bloccato in attesa sul Semaforo.

## 5 Conclusioni e Sviluppi Futuri

L'Hands-On 11 offre la prova empirica definitiva che lo scambio di memoria in ambito multithreading non può mai assumere la connotazione dell'idempotenza se non

attraverso un sistema regolamentato di Mutex, Monitor o Semafori. Si è osservato come l'implementazione pratica del Mutex garantisca sicurezza, sacrificando inevitabilmente il parallelismo perfetto nel punto interessato.

**Sviluppi futuri:**

- **Deadlock Prevention**: Esplorare l'implementazione di sistemi di priorità e ordine logico formale nell'acquisizione di mutex multipli paralleli ai fine da evitare che due thread si blocchino permanentemente su lock ciclici (Deadlocks).
- **Rwlocks (Read-Write Locks)**: Implementare soluzioni avanzate che consentano l'accesso parallelo e contemporaneo alla lettura dei dati ma rendano vincolante la mutua esclusione solo per la scrittura, ottimizzando fortemente contesti ove è richiesta un'informazione aggiornata ma raramente modificata.